CRITICAL: Degraded Mode Not Falling Back to Live Providers

Problem

When Gemini is unavailable, Karen shows:

```
Requested provider gemini was unavailable; Karen continued in degraded
mode.
Service temporarily unavailable. Please try again shortly.
```

Expected Behavior: Should fall back to vLLM or Transformers and provide a LIVE response.

Actual Behavior: Shows static "unavailable" message instead of using fallback providers.

Root Cause Analysis

The system is entering "degraded mode" but NOT executing the fallback chain:

```
gemini (unavailable) → builtin_vllm → builtin_transformers → fallback
```

Metadata Shows

```
{
  "provider": "gemini",
  "model": "gemini-2.5-flash",
  "source": "requested_model",
  "status": "degraded mode",
  "latency": "0.35s"
}
```

PROF

Problem: Metadata still shows `provider: "gemini"` instead of the actual fallback provider that should have answered.

This Violates Audit Requirements

From the vLLM audit (Task 6):

> When Gemini fails and Karen successfully answers through vLLM, metadata should show:
>
> ```
> {
>   "requested_provider": "gemini",
> ```

```
    "actual_provider": "vllm",
    "runtime_engine": "vllm",
    "response_source": "live_model",
    "fallback_level": 1,
    "degraded_mode": true,
    "degradation_reason": "requested_provider_unavailable"
  }
```

Current behavior: System stops at "degraded mode" without trying fallback providers.

Investigation Steps

1. Check Backend Logs

```
# Docker
docker compose logs api | grep -A 10
"gemini.*unavailable\|fallback\|degraded"

# Local
tail -100 logs/karen_api.log | grep -A 10
"gemini.*unavailable\|fallback\|degraded"
```

Look for:

- Provider selection attempts
- Fallback chain execution
- vLLM/Transformers invocation
- Error messages

2. Check Provider Health

PROF

```
# Check if vLLM is available
curl http://localhost:8001/health

# Check if Transformers is available
curl http://localhost:8000/api/health/providers | jq
'.providers.builtin_transformers'
```

3. Test Direct vLLM Request

```
# Bypass Gemini, request vLLM directly
curl -X POST http://localhost:8000/api/chat \
  -H "Content-Type: application/json" \
  -d '{
    "message": "test",
```

```
    "provider": "vllm",
    "model": "auto"
  }' | jq
```

Expected: Should work if vLLM is configured.

Possible Root Causes

Cause 1: Fallback Chain Not Executing

File: `src/ai_karen_engine/services/models/routing/llm_router_service.py`

The router may be stopping at "degraded mode" without trying fallback providers.

Check:

```
# Around line 2089-2107
RUNTIME_DEGRADED_FALLBACK_ORDER = (
    "builtin_vllm",
    "builtin_transformers",
    "fallback",
)
```

Verify: Is this fallback order actually being used when Gemini fails?

Cause 2: vLLM Not Configured

Check:

```
# Is VLLM_BASE_URL set?
echo $VLLM_BASE_URL

# Is vLLM server running?
curl http://localhost:8001/health
```

PROF

If vLLM is not configured, the system should fall back to Transformers.

Cause 3: Emergency Static Response Triggered Too Early

File: `src/ai_karen_engine/core/runtime/degraded_mode.py`

The system may be returning emergency static response instead of trying fallback providers.

Check: Lines around `generate_degraded_mode_response()`

Cause 4: Provider Health Check Failing All Providers

File: `src/ai_karen_engine/core/operations/health_checker.py`

All providers may be marked as unhealthy, causing immediate degraded mode.

Check:

```
curl http://localhost:8000/api/health/providers | jq
```

Immediate Fix Steps

Step 1: Enable Debug Logging

File: `.env` or environment

```
# Add these
LOG_LEVEL=DEBUG
KARI_DEBUG=true
```

Restart services:

```
docker compose restart api
```

Step 2: Test Fallback Manually

```
# test_fallback.py
import asyncio
from ai_karen_engine.services.models.routing.llm_router_service import
LLMRouter

async def test():
    router = LLMRouter()

    # Simulate Gemini failure
    request = {
        "message": "test",
        "provider": "gemini",
        "model": "gemini-2.5-flash"
    }

    # This should fall back to vLLM
    result = await router.select_provider(request)
    print(f"Selected provider: {result}")

asyncio.run(test())
```

PROF

Step 3: Check Fallback Configuration

File: `src/ai_karen_engine/config/config_manager.py`

```
# Around line 247-249
"fallback_chain": [
    "builtin_vllm",
    "builtin_transformers",
    "openai",
    "gemini",
    "deepseek",
    "huggingface"
],
```

Verify: Is this configuration being loaded correctly?

Required Fixes

Fix 1: Ensure Fallback Chain Executes

File: `src/ai_karen_engine/services/models/routing/llm_router_service.py`

Around the provider selection logic, ensure fallback is attempted:

```
async def select_provider(self, request):
    # Try requested provider
    try:
        result = await self._try_provider(requested_provider)
        if result:
            return result
    except Exception as e:
        logger.warning(f"Provider {requested_provider} failed: {e}")

    # CRITICAL: Try fallback chain
    for fallback_provider in self.RUNTIME_DEGRADED_FALLBACK_ORDER:
        try:
            logger.info(f"Trying fallback provider:
{fallback_provider}")
            result = await self._try_provider(fallback_provider)
            if result:
                return result
        except Exception as e:
            logger.warning(f"Fallback provider {fallback_provider}
failed: {e}")

    # Only return static response if ALL providers failed
    return self._emergency_static_response()
```

PROF

Fix 2: Update Metadata to Show Actual Provider

When fallback succeeds, metadata MUST show:

```
{
    "requested_provider": "gemini",
    "actual_provider": "builtin_vllm",  # ← The provider that actually
answered
    "runtime_engine": "vllm",
    "response_source": "live_model",    # ← NOT "emergency_static"
    "degraded_mode": true,
    "fallback_level": 1
}
```

Fix 3: Don't Return Static Message If Fallback Works

Current (WRONG):

```
Gemini unavailable → Show "Service temporarily unavailable"
```

Correct:

```
Gemini unavailable → Try vLLM → vLLM answers → Show vLLM response
```

Testing After Fix

Test 1: Gemini Unavailable, vLLM Works

PROF

```
# Ensure Gemini API key is invalid or removed
unset GEMINI_API_KEY

# Request should fall back to vLLM
curl -X POST http://localhost:8000/api/chat \
  -H "Content-Type: application/json" \
  -d '{
    "message": "Say hello",
    "provider": "gemini",
    "model": "gemini-2.5-flash"
  }' | jq
```

Expected:

```
{
  "answer": "Hello! [actual generated text from vLLM]",
  "metadata": {
    "llm": {
      "requested_provider": "gemini",
      "actual_provider": "builtin_vllm",
      "response_source": "live_model",
      "degraded_mode": true
    }
  }
}
```

Test 2: All Providers Unavailable

```
# Stop vLLM server
# Remove all API keys
# Request should show emergency static

curl -X POST http://localhost:8000/api/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "test"}' | jq
```

Expected:

```
{
  "answer": "I apologize, but all AI providers are currently
unavailable...",
  "metadata": {
    "llm": {
      "actual_provider": "emergency",
      "response_source": "emergency_static",
      "degraded_mode": true
    }
  }
}
```

PROF

Priority

🔴 **CRITICAL** - This breaks the entire fallback system.

The vLLM audit proved vLLM is wired correctly, but the fallback chain is not executing.

Related Files

- **Router:** `src/ai_karen_engine/services/models/routing/llm_router_service.py`
- **Degraded Mode:** `src/ai_karen_engine/core/runtime/degraded_mode.py`

- **Control Plane:** `src/ai_karen_engine/core/runtime/chat_runtime_control_plane.py`
- **Orchestrator:** `src/ai_karen_engine/llm_orchestrator.py`

Next Steps

1. ✅ Enable debug logging
2. ✅ Check backend logs for fallback attempts
3. ✅ Verify vLLM is available
4. ✅ Test direct vLLM request
5. 🔄 Fix fallback chain execution
6. 🔄 Update metadata to show actual provider
7. 🔄 Test Gemini → vLLM fallback
8. 🔄 Verify no static response when fallback works

Created: 2026-04-27
Status: Investigation required
Impact: HIGH - Breaks fallback system
Estimated Fix Time: 2-4 hours